



FOR JAVA DEVELOPERS MOVING TO NESTJS

One line. Four traps.

Spring Boot to NestJS, episode 22: calling other APIs

A one-line HTTP call hides four decisions: when it gives up, whether it ran, what counts as failure, and whether it tries again.

Against an API that answers after ten seconds, `RestClient`, `@nestjs/axios` and the new `@nestjs/http-client` all waited the full ten seconds by default.

With a one second timeout the new client gave up after 3.2 seconds with three hits: the timeout is per attempt and retries are on by default.

The downstream saw the caller hang up at one second and still finished its work. A timeout bounds your wait, not theirs.

In scope

- `RestClient` and `WebClient`
- `@nestjs/axios` `HttpService`
- `@nestjs/http-client` `HttpClient`
- Timeouts, laziness, errors, retries

Not in scope

- Circuit breakers (next episode)
- Streaming large bodies

At the time of recording the Nest docs describe `@nestjs/http-client` 0.0.1; `@nestjs/axios` remains available.

01 The measurements

DEFAULT, API TAKES 10 s	all three waited 10.0 s
1 s TIMEOUT	RestClient 1.2 s; axios 1.0 s; http-client 3.2 s, 3 hits
NEVER SUBSCRIBED / AWAITED	WebClient 0, HttpService 0; RestClient 1, HttpClient 1
GET A 500	error on all three; fetch resolves ok false
FAILS ONCE, THEN 200	RestClient, axios: fail; http-client: 200 after 2 hits
PUT / POST A 500, http-client	PUT 3 hits, POST 1

One downstream server, counting hits per path.

02 Set the timeout on every client

Spring: `spring.http.clients.read-timeout`. @nestjs/axios: `HttpModule.register({ timeout })`.
@nestjs/http-client: `HttpClientModule.register({ timeout })`, applied per attempt.

```
application.properties
```

```
spring.http.clients.read-timeout=1s
```

```
app.module.ts
```

```
HttpModule.register({ timeout: 1000 }),  
HttpClientModule.register({ timeout: '1s' }),
```

http-client, timeout 1s, API takes 10 s: `HttpTimeoutError` after 3.2 s, three hits

Run on a real machine, 2026-09-30, by `scripts/verify-http.sh` in the course repository.

03 Whether it ran

`HttpService` returns an `Observable`: nothing is sent until you subscribe, for example with `firstValueFrom`. The new `HttpClient` returns a promise and sends immediately.

```
payments.client.ts
```

```
const res = await firstValueFrom(this.http.get(`${base}/payments/${id}`));  
const { data } = await this.client.get(`/payments/${id}`);
```

`HttpService.get()` without `firstValueFrom`: zero requests received

Run on a real machine, 2026-09-30, by `scripts/verify-http.sh` in the course repository.

Before you trust an HTTP call

1. Is there a timeout, and is it per call or per attempt?
2. A timeout does not cancel the other side's work
3. Is the call lazy? Is anything subscribing?
4. What does a 500 do: throw, or resolve?
5. Does the client retry by default, and which methods?
6. Only retry what the endpoint makes idempotent

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-http.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS HTTP client https://docs.nestjs.com/techniques/http-module

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.